

*"If you cannot see it, you cannot operate it
— logs, metrics, and traces as one story."*

SUPPLEMENTARY · MODULE 9

Observability

Instrument Spring Boot for production: structured logs, Micrometer metrics, distributed tracing, and the alerts that turn a banking incident into a known playbook.





DAY 4 OBJECTIVES AND CAPSTONE PROGRESS



DAY 4 OBJECTIVES



Implement Asynchronous Communication

Use Kafka to enable event-driven messaging between services.



Event-Driven Architecture

Design and implement domain events and event flows.



External System Integration

Integrate with an external Fraud Detection service via REST API.



Reliability & Observability

Ensure reliability with retries, DLQ, idempotency and monitor with metrics and logs.



Testing & Validation

Write integration and contract tests to validate the complete flow.



CAPSTONE PROGRESS



Account Service

User and account management

✓ COMPLETE



Transaction Service

Create and manage transactions

✓ COMPLETE



Notification Service

Send alerts and notifications

✓ COMPLETE



Security & Resilience

OAuth2, Resilience4j, Rate Limiting, Config, Observability

✓ COMPLETE



Event-Driven & Async Communication

Kafka setup, events, producers and consumers

🔄 IN PROGRESS



External Integration (Fraud Service)

REST API integration and response handling

🔄 IN PROGRESS



End-to-End Validation

End-to-end flow, tests and monitoring

⌚ PENDING

CAPSTONE ARCHITECTURE PROGRESSION



Client

DONE



API Gateway

DONE



Account Service

DONE



Transaction Service

DONE



Kafka

IN PROGRESS



Notification Service

DONE



Fraud Detection

IN PROGRESS



Monitoring & Observability

PENDING

MONITORING VS. OBSERVABILITY

Monitoring answers known unknowns with dashboards and thresholds; observability explores unknown unknowns through logs, metrics, and traces.



MONITORING: FOCUS ON KNOWN UNKNOWN

- Collect metrics -> alert on thresholds -> investigate, resolve, notify
- Best for detecting known issues, SLA tracking, capacity planning, compliance



OBSERVABILITY: FOCUS ON UNKNOWN UNKNOWN

- Collect logs, metrics, traces -> correlate & explore -> understand, decide, improve
- Best for exploring unknown issues, complex behavior, and root cause analysis

KEY TAKEAWAY Monitoring answers known unknowns with dashboards and thresholds; observability explores unknown unknowns through logs, metrics, and traces.

FROM WORKING SERVICES TO OPERABLE SERVICES

WORKING SERVICES

"It works on my machine"



FUNCTIONAL FOCUS

Focus on features and happy path functionality



MANUAL OPERATIONS

Manual deployment, configuration and troubleshooting



LIMITED RESILIENCE

Little to no handling of failures, retries or timeouts



NO OBSERVABILITY

Limited logging, no metrics, no tracing, hard to diagnose



BASIC TESTING

Unit tests only, limited integration and end-to-end testing



OPERABLE SERVICES

"It works in production, for everyone"



RELIABILITY BY DESIGN

Built with resilience patterns: retry, circuit breaker, timeout, idempotency



AUTOMATED OPERATIONS

Externalized configuration, automated deployment and self-service operations



BUILT-IN RESILIENCE

Graceful handling of failures, fallbacks, bulkheads and rate limiting



FULL OBSERVABILITY

Logs, metrics, tracing, dashboards and alerting in place



COMPREHENSIVE TESTING

Unit, integration, contract and end-to-end tests with Testcontainers

ENABLERS OF OPERABLE SERVICES



SECURITY

OAuth2, JWT, Input Validation



RESILIENCE

Resilience4j, Circuit Breaker, Retry, Timeout



RATE LIMITING

Protect services and upstreams



CONFIGURATION

Spring Cloud Config, Profiles, Git-backed



OBSERVABILITY

Actuator, Prometheus, Grafana, Tracing



TESTING

Integration tests, Testcontainers



AUTOMATION

CI/CD, Health checks, Runbooks



OUTCOME: Reliable, Secure, Scalable and Maintainable Services in Production

THE THREE PILLARS OF OBSERVABILITY · APPLICATION LOGGING STRATEGY



LOGS: WHAT HAPPENED?

- Timestamped records of discrete events; high cardinality, unstructured, stored as text
- **Example:** "Error connecting to database at 10:15:32 from service OrderService"



WHAT TO LOG

- Operational events (startup, requests, business operations)
- Diagnostics (errors, validation failures), security & audit, performance



METRICS & TRACES

- **Metrics:** numeric measurements over time -- ideal for alerting and dashboards
- **Traces:** follow a request across services -- reveal latency, bottlenecks, dependencies



HOW TO LOG IT

- Use structured (JSON) logging with consistent log levels
- Include correlation ID, user ID, service, and environment on every entry
- Never log sensitive data -- mask or exclude PII, credentials, tokens

KEY TAKEAWAY Logs give detailed context, metrics show trends and health, and traces reveal dependencies and root causes -- each pillar alone is incomplete. • Operational events, diagnostics, security/audit, and performance signals each need a place in a well-defined logging plan.

STRUCTURED AND CENTRALIZED LOGGING

Fluent Bit/Filebeat ship logs from every microservice into Elasticsearch/Logstash/Kibana, where correlationId ties them all back together.



CENTRALIZED LOGGING ARCHITECTURE

- Microservices -> Log Shipper/Agent (Fluent Bit/Filebeat) -> Centralized Log Store (Elasticsearch)
- -> **Users & Tools:** search & analyze, dashboards, alerts, audit & compliance



BEST PRACTICES

- Log in structured JSON consistently; include correlationId/tracelId in every log
- Centralize logs, set retention policies, and mask sensitive data

KEY TAKEAWAY Fluent Bit/Filebeat ship logs from every microservice into Elasticsearch/Logstash/Kibana, where correlationId ties them all back together.



STRUCTURED LOGGING AND CORRELATION IDS



Structured logging provides machine-readable logs. Correlation IDs help trace a single request across multiple services.

WHY STRUCTURED LOGGING?



Easier Search & Analysis

Structured logs are easy to parse, search and analyze.



Better Observability

Enables dashboards, alerts and insights.



Machine Readable

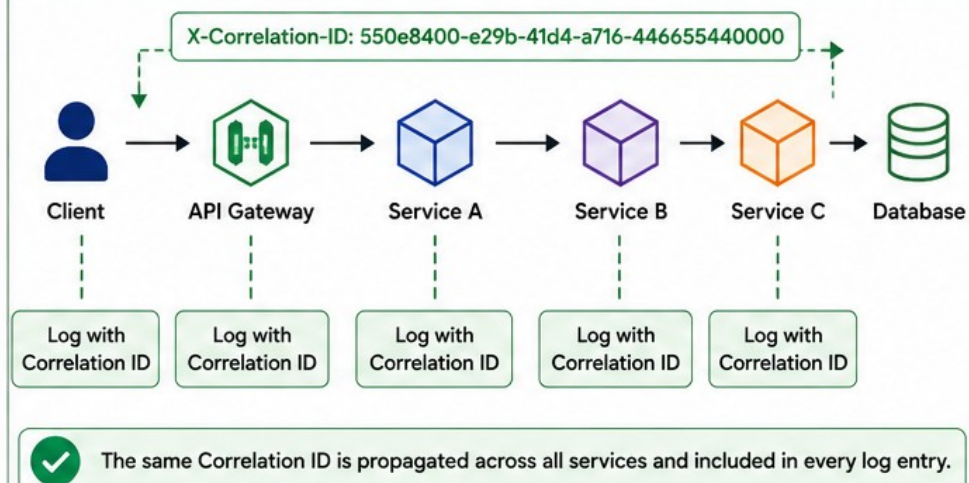
Logs in JSON format are easy for tools to consume.



Consistent & Standardized

Uniform log structure across all services.

CORRELATION ID FLOW



HOW TO IMPLEMENT

1

Generate / Extract ID

Generate a Correlation ID at the entry point (e.g., API Gateway) or extract from incoming request header.

2

Propagate

Add the Correlation ID to MDC (Mapped Diagnostic Context) and outgoing requests (e.g., headers).

3

Log with ID

Include the Correlation ID in every log entry using MDC.

4

Clear MDC

Clear the MDC after request completion to avoid leakage.

STRUCTURED LOG EXAMPLE (JSON)

```
{
  "timestamp": "2025-05-15T10:15:30.123Z",
  "level": "INFO",
  "service": "transaction-service",
  "traceId": "550e8400-e29b-41d4-a716-446655440000",
  "spanId": "a1b2c3d4e5f6",
  "message": "Transaction created successfully",
  "http": {
    "method": "POST",
    "path": "/api/transactions",
    "status": 201
  },
  "userId": "user-12345",
  "durationMs": 152,
  "event": "transaction.create"
}
```

KEY FIELDS TO INCLUDE

	timestamp	When the log was recorded
	level	Log level (INFO, WARN, ERROR)
	service	Name of the service
	traceId / correlationId	Unique ID to trace the request
	spanId	(Optional) For distributed tracing
	message	Log message
	http	HTTP method, path, status
	userId	User identifier (if applicable)
	durationMs	Request processing time
	event	Business event name

BEST PRACTICES



Use a standard header name
e.g., X-Correlation-ID



Propagate the ID in all service-to-service calls



Log in JSON format



Avoid logging sensitive data



Clear MDC after request completion



Use logs for monitoring, alerting and debugging

METRICS AND TIME-SERIES MONITORING · PROMETHEUS METRICS MODEL · SPRING BOOT ACTUATOR AND MICROMETER



HOW IT WORKS

- Your Application (Spring Boot) -> Prometheus/Micrometer collection -> Time-Series Storage
- -> Grafana visualization -> Dashboards & Alerts



KEY METRIC CATEGORIES

- Application performance, resource utilization, business metrics
- Service health, and SLOs and limits (latency/error thresholds)



CORE CONCEPTS

- **Time series:** a sequence of timestamped values identified by metric name and labels
- Labels give dimensional context; samples are the actual recorded values
- All time series stored in the Prometheus TSDB



METRIC TYPES

- Counter (monotonically increasing), Gauge (up or down), Histogram/Summary (distributions)



WHAT THEY PROVIDE

- **Actuator:** production-ready endpoints (health, info, metrics, loggers, threaddump)
- **Micrometer:** a metrics facade that instruments code and publishes to Prometheus, Grafana, Datadog, and more



BEST PRACTICES

- Expose only the endpoints you need; secure Actuator endpoints
- Use meaningful metric names and tags; monitor SLOs with actionable dashboards

KEY TAKEAWAY Spring Boot instruments with Micrometer, Prometheus stores the time series, and Grafana turns it into dashboards and alerts. • `http_requests_total{method="GET", status="200", service="order"} 1027` shows exactly how metric name, labels, and sample value combine.

- `/actuator/health`, `/actuator/metrics`, and `/actuator/prometheus` turn a Spring Boot service into an observable system with two dependencies.

Metrics Basics



What is a Metric?

A metric is a numerical measurement captured over time to describe the behavior and health of a system.



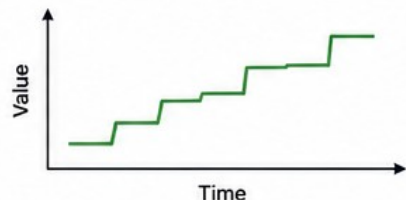
Key Characteristics

- ✓ Numeric
- ✓ Captured over time
- ✓ Has a Name and Tags (labels)
- ✓ Used for monitoring and alerting

Metric Types

1. Counter

A monotonically increasing value that only goes up.

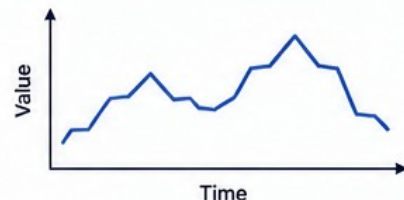


Example:

`http_requests_total`, `errors_total`

2. Gauge

A value that can go up and down.

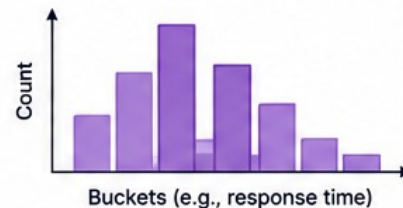


Example:

`jvm_memory_used_bytes`,
`cpu_usage_percent`

3. Histogram

Records the distribution of observations into buckets.



Example:

`http_server_requests_seconds`

4. Summary

Similar to Histogram, calculated client-side with quantiles.

Quantile	Value (ms)
0.50 (P50)	120
0.90 (P90)	250
0.99 (P99)	600

Example:

`http_server_requests_seconds`



Metric Naming Best Practices

- Use clear, descriptive names
- Use consistent naming conventions
- Include unit in the name (e.g., `_seconds`, `_bytes`, `_total`)
- Use labels/tags for dimensions, not in the name

Good: `http_server_requests_seconds_total`

Avoid: `HTTPRequests`, `reqTime`



Common Use Cases

- Monitor system health and performance
- Understand usage patterns
- Set alerts and SLOs
- Capacity planning and troubleshooting



Remember:

Good metrics provide visibility, drive insights, and enable proactive operations.



Application

Emits
Metrics



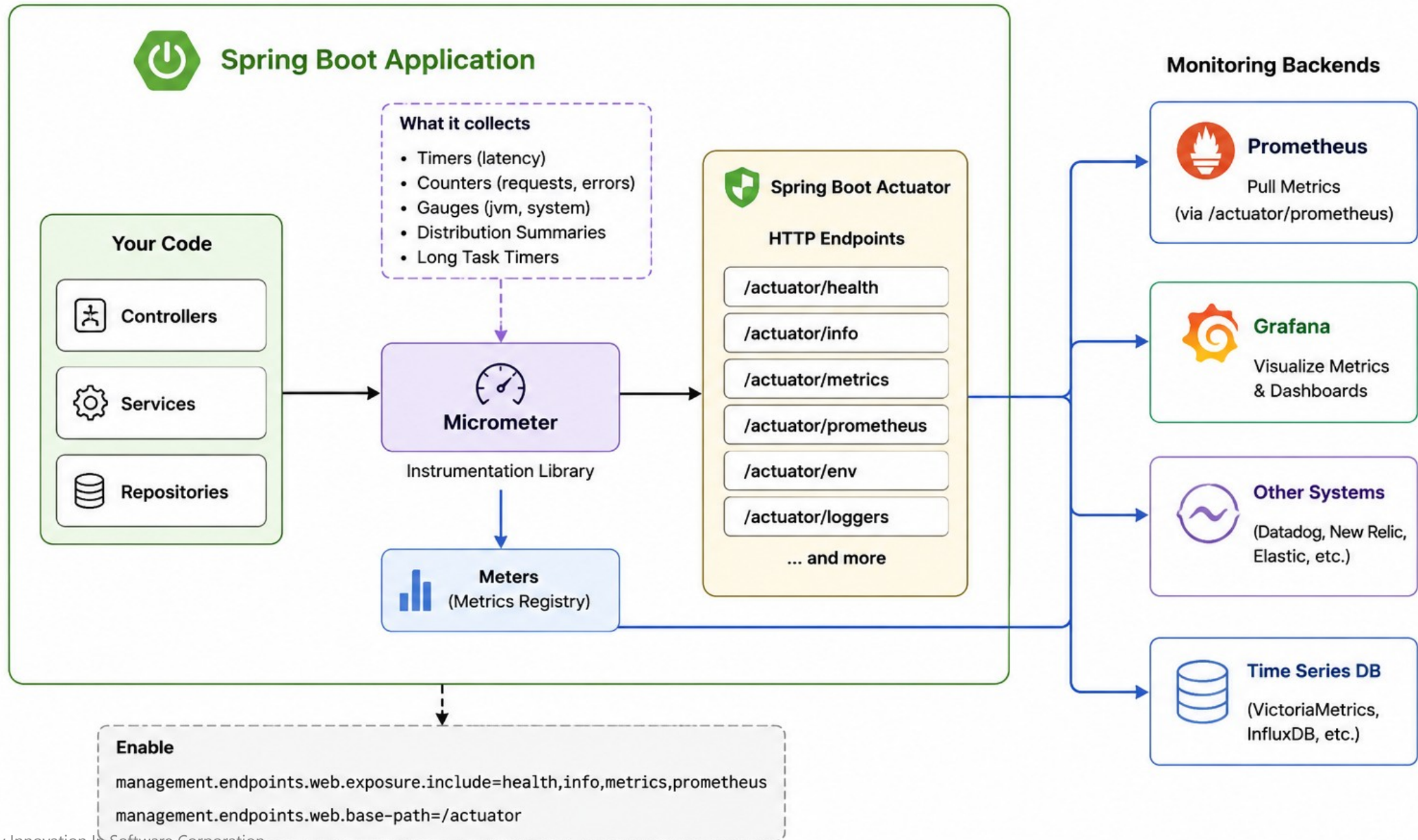
Metrics System
Collects, stores,
and processes
metrics

Visualize & Alert



Dashboards / Alerts
Charts, dashboards,
and alerts for
observability

Actuator and Micrometer



CUSTOM BUSINESS AND TECHNICAL METRICS · GRAFANA DASHBOARD DESIGN · DISTRIBUTED TRACING



TYPES OF CUSTOM METRICS

- **Counter:** a monotonically increasing value (e.g., orders.placed)
- **Timer:** records duration and percentiles (e.g., payment.processing.time)
- Gauge and Distribution Summary for point-in-time values and value distributions



LAYOUT TIPS

- KPIs at the top, trends in the middle (time series), details at the bottom (tables)
- Use variables for environment/service/region filters; group panels into rows



NAMING & TAGGING GUIDELINES

- Use dot-separated, lowercase, consistent metric names
- **Good tags:** service, region, status, channel; avoid high-cardinality tags like user.id



WHY DISTRIBUTED TRACING

- End-to-end visibility of a request's full path across services
- Faster root cause analysis; visualizes service dependencies
- Reduces MTTR and validates SLOs and user experience



DESIGN PRINCIPLES

- Know your audience and focus on the goals/questions the dashboard should answer
- Favor clarity over complexity; use time ranges that fit the use case
- Highlight issues and make insights actionable



TRACE + METRICS + LOGS = OBSERVABILITY

- Traces (structure) + Metrics (quantitative) + Logs (context) = actionable insight
- Propagate trace-id/span-id across every service and messaging boundary

KEY TAKEAWAY A Counter for orders.placed and a Timer for payment.processing.time turn business outcomes and technical health into first-class metrics. • Request rate, error rate, p95 latency, availability, and saturation as top-row KPIs give an instant read on service health before anyone drills in. • A request flowing through API Gateway, Order, Payment, and Inventory services becomes one connected trace, with each hop as a timed span.

TRACE AND SPAN CONCEPTS

Understanding the Building Blocks of Distributed Tracing



WHAT IS A TRACE?



A Trace represents the complete journey of a request as it flows through a distributed system. It has a unique Trace ID.

WHAT IS A SPAN?



A Span represents a single unit of work within a trace. It has a start time, end time, and additional metadata. It has a unique Span ID.

KEY RELATIONSHIPS



A trace contains one or more spans.



Spans form a hierarchy. A span may have a parent span.



Context (Trace ID, Span ID) is propagated between services.

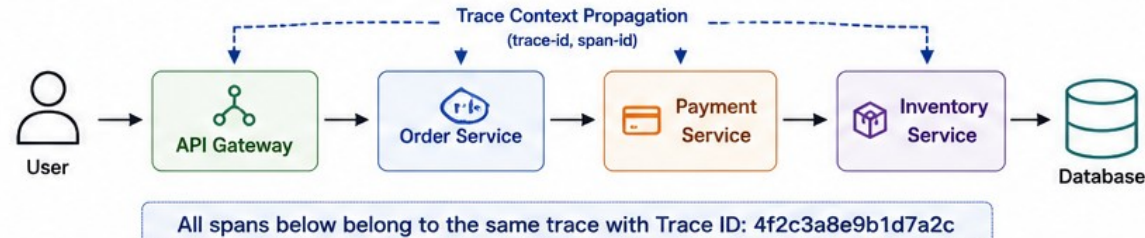


Spans have timing information to show duration and latency.

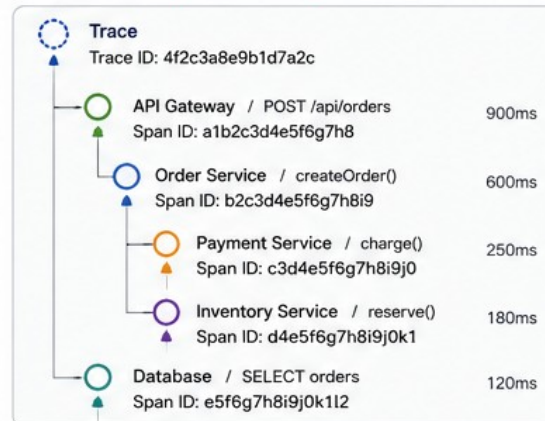
WHY IT MATTERS

- ✓ End-to-end visibility of requests
- ✓ Identify latency and bottlenecks
- ✓ Understand service dependencies
- ✓ Improve reliability and user experience
- ✓ Faster troubleshooting and RCA

EXAMPLE: A TRACE ACROSS MICROSERVICES



TRACE VIEW (HIERARCHY)



SPAN ANATOMY

trace-id	span-id	parent-span-id	name	start-time	end-time	attributes (tags)	events	status
Unique ID for the entire trace (consistent across all spans)	Unique ID for this span	ID of the parent span (empty for root span)	Operation or function name (e.g., GET /orders)	When the span started	When the span ended	Key-value metadata (e.g., http.method, status.code)	Important moments within the span	Status code (OK, Error) and message

CONTEXT PROPAGATION (EXAMPLE HTTP HEADERS)

```
traceparent: 00-4f2c3a8e9b1d7a2c7c9d3a1b4e6f8d12-00f067aa0ba902b7-01
tracestate: vendor=abc123,env=prod
```

The traceparent header carries Trace ID and Span ID so downstream services can continue the trace.

SPAN TYPES



Server Span
Represents an incoming request handled by a service.



Client Span
Represents an outgoing request made by a service.



Internal Span
Represents work done within a service (e.g., computation, DB query, cache lookup).

BEST PRACTICES

- ✓ Name spans meaningfully (use operations, not just URLs)
- ✓ Keep spans small and focused
- ✓ Set useful attributes and events
- ✓ Propagate context across all service boundaries
- ✓ Sample smartly to balance visibility and cost
- ✓ Monitor traces with logs and metrics for full observability



KEY TAKEAWAY

A trace shows the full journey. Spans show the steps. Together they reveal the story behind every request.

CONTEXT PROPAGATION

W3C Trace Context headers (traceparent, tracestate) carry the trace-id and span-id across HTTP, messaging, and gRPC boundaries alike.



HOW CONTEXT FLOWS

- **HTTP/REST:** W3C Trace Context headers (traceparent, tracestate)
- **Messaging (Kafka in Lab 2):** correlation id injected into message headers
- **gRPC and async events:** context carried in metadata or event headers



THE PROPAGATION LIFECYCLE

- 1. Create:** the first service starts a trace and generates the trace and span identifiers
- 2. Inject:** those identifiers are written into outbound headers before the call leaves
- 3. Transmit:** the headers travel with the request over HTTP or alongside the message
- 4. Extract:** the receiving service reads the identifiers back out of the incoming request
- 5. Continue:** it opens a new span linked to the same trace, keeping the path connected end to end

KEY TAKEAWAY W3C Trace Context headers (traceparent, tracestate) carry the trace-id and span-id across HTTP, messaging, and gRPC boundaries alike.

OPENTELEMETRY OVERVIEW

The Open Standard for Observability



WHAT IS OPENTELEMETRY?



OpenTelemetry (OTel) is an open-source observability framework for generating, collecting, and exporting telemetry data.

It provides a vendor-neutral standard for:

- Instrumentation
- Data collection
- Context propagation
- Export to multiple backends

KEY BENEFITS



Vendor Neutral

Send telemetry data to any observability backend.



Standardized

Unified APIs, data models, and context across tools.



End-to-End Visibility

Correlate logs, metrics, and traces across your entire system.



Flexibility

Use only what you need: traces, metrics, logs—or all three.



Ecosystem Growth

Large, active community and broad industry adoption.



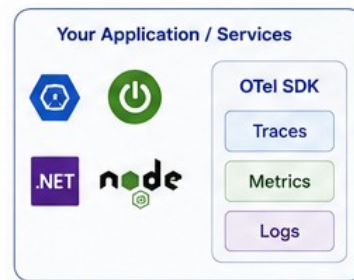
WHY IT MATTERS

OpenTelemetry enables consistent observability across diverse technologies and environments, improving troubleshooting, performance, and reliability.

HOW OPENTELEMETRY WORKS

1. INSTRUMENTATION

Your application or service emits telemetry data.



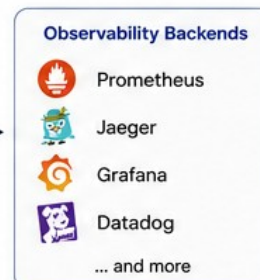
2. COLLECTION

The Collector receives, processes, and exports telemetry data.



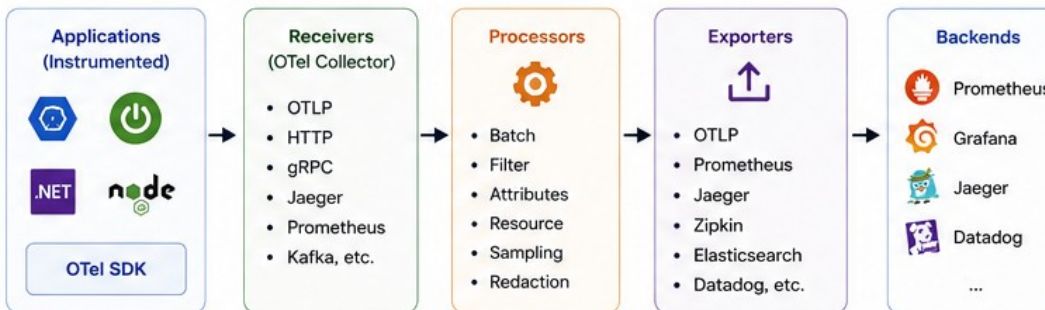
3. EXPORT

Telemetry is exported to one or more backends.



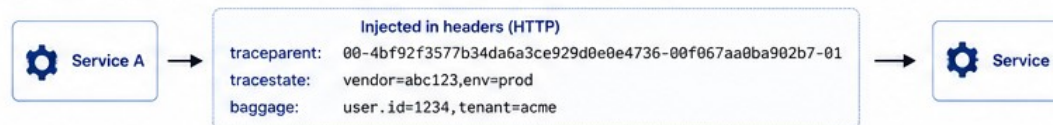
Context Propagation (trace-id, span-id, baggage)

OPENTELEMETRY ARCHITECTURE



CONTEXT PROPAGATION

OpenTelemetry uses W3C Trace Context to propagate context across service boundaries.



THREE SIGNALS OF OBSERVABILITY



TRACES

Capture the flow of requests as they travel through your system. Helps answer "Why is it slow?"



METRICS

Numerical measurements about your system over time. Helps answer "What is the current state?"



LOGS

Discrete events recorded by your applications. Helps answer "What happened?"

CORE COMPONENTS



SDKs

Language-specific libraries to instrument your applications.



Collector

Vendor-agnostic service that receives, processes, and exports data.



Exporters

Send data to one or more observability backends.



Integrations

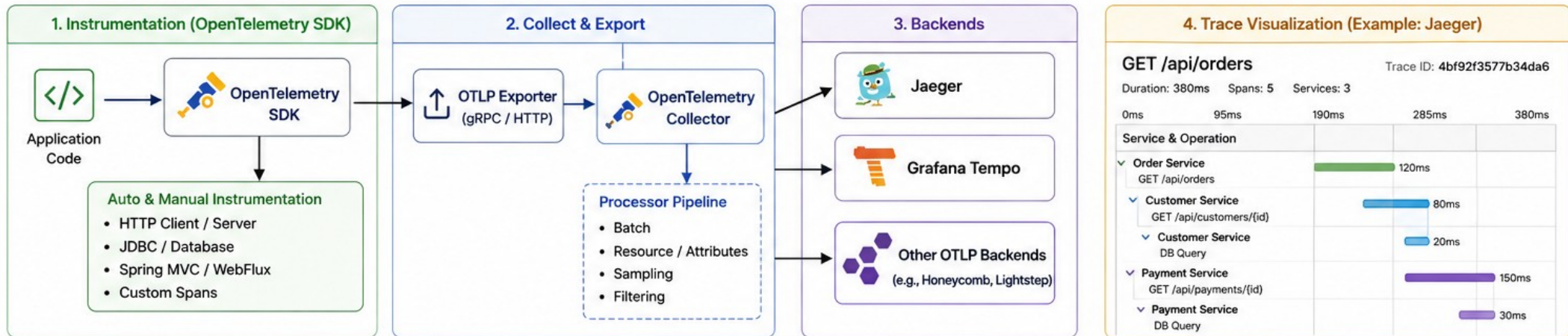
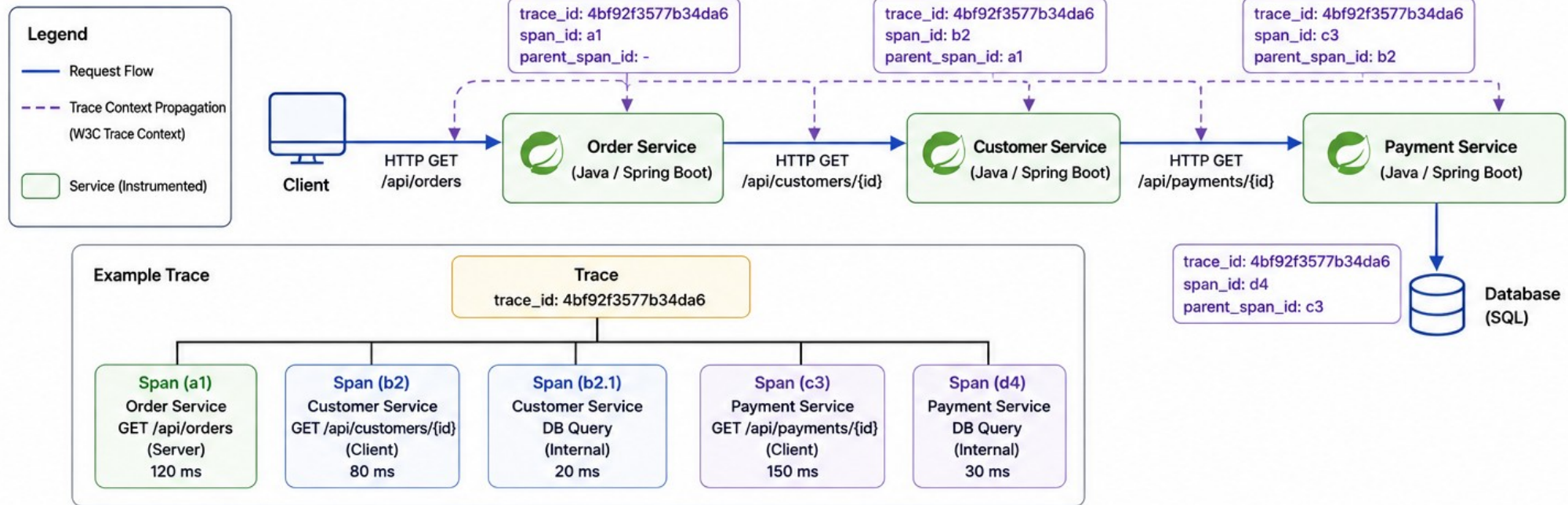
Pre-built instrumentation for popular frameworks, libraries, and platforms.

WIDELY INTEGRATED WITH



... and many more technologies

Distributed Tracing Demonstration (OpenTelemetry)



Key Benefits



End-to-end visibility
across services



Faster root cause
analysis



Performance
optimization



Understand latency
breakdown



Correlate logs, metrics
and traces

CORRELATING LOGS, METRICS, AND TRACES · HEALTH CHECKS AND READINESS SIGNALS



HOW CORRELATION WORKS

1. A request is made and assigned a unique trace-id
2. The trace-id is propagated across all services
3. Services emit logs, metrics, and spans with the same context
4. Visualization tools (Grafana) link the data using correlation fields and time



THREE TYPES OF HEALTH SIGNALS

- **Liveness probe:** "Is my app running?" -- failure triggers a container restart
- **Readiness probe:** "Is my app ready to serve traffic?" -- failure removes the pod from Service endpoints
- **Startup probe:** "Is my app started up?" -- protects slow-starting apps during initialization

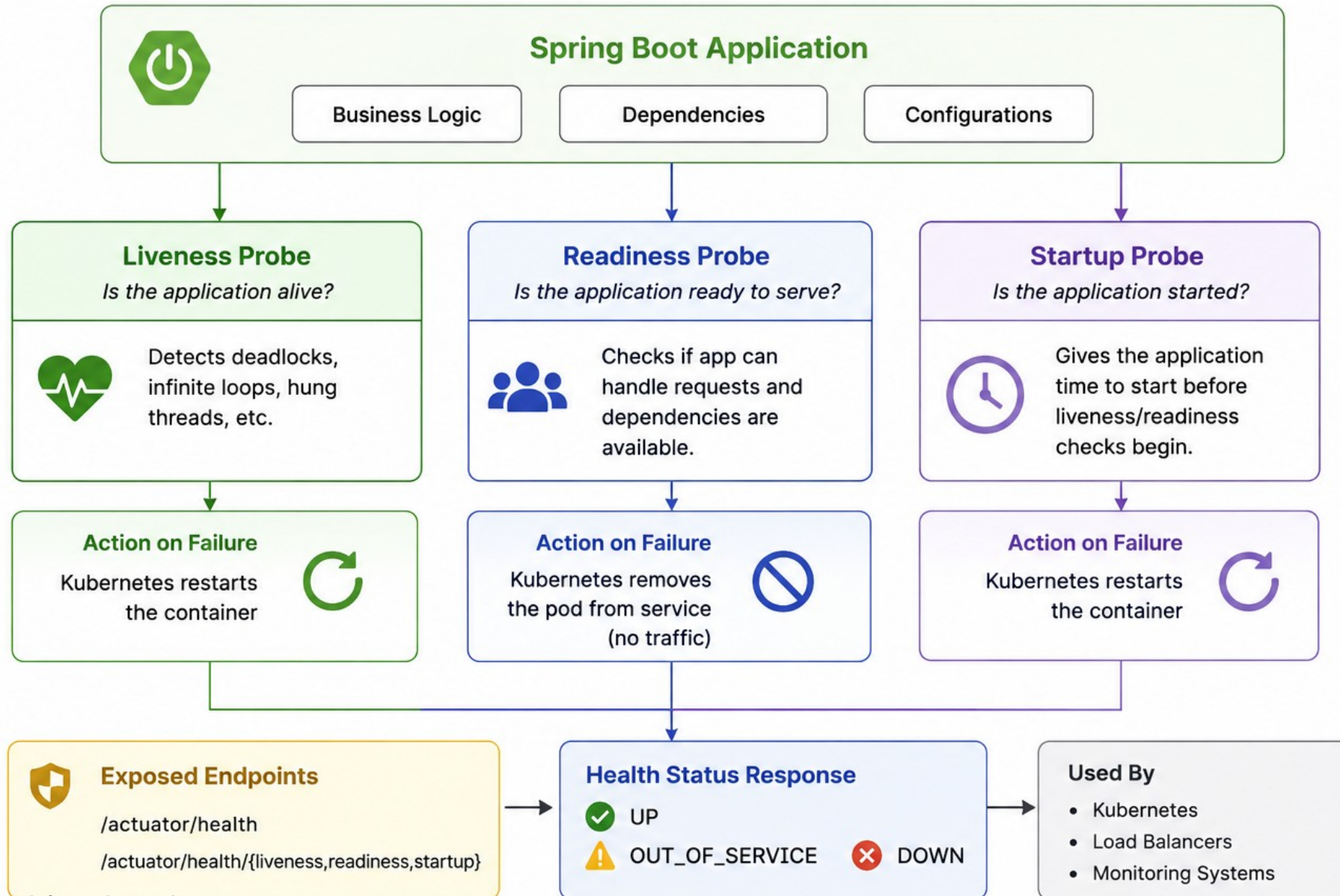


SPRING BOOT ACTUATOR EXAMPLE

- GET /actuator/health/liveness and /actuator/health/readiness both return {"status": "UP"}

KEY TAKEAWAY Jumping from a metrics spike to the exact trace to the precise error log -- all linked by one trace-id -- is what turns signals into a root cause. • Liveness, readiness, and startup probes each answer a different question, and together they keep unhealthy pods off the traffic path.

Health Checks - Liveness, Readiness, Startup



LIVENESS VS. READINESS PROBES

Observability
The Three Pillars



Logs



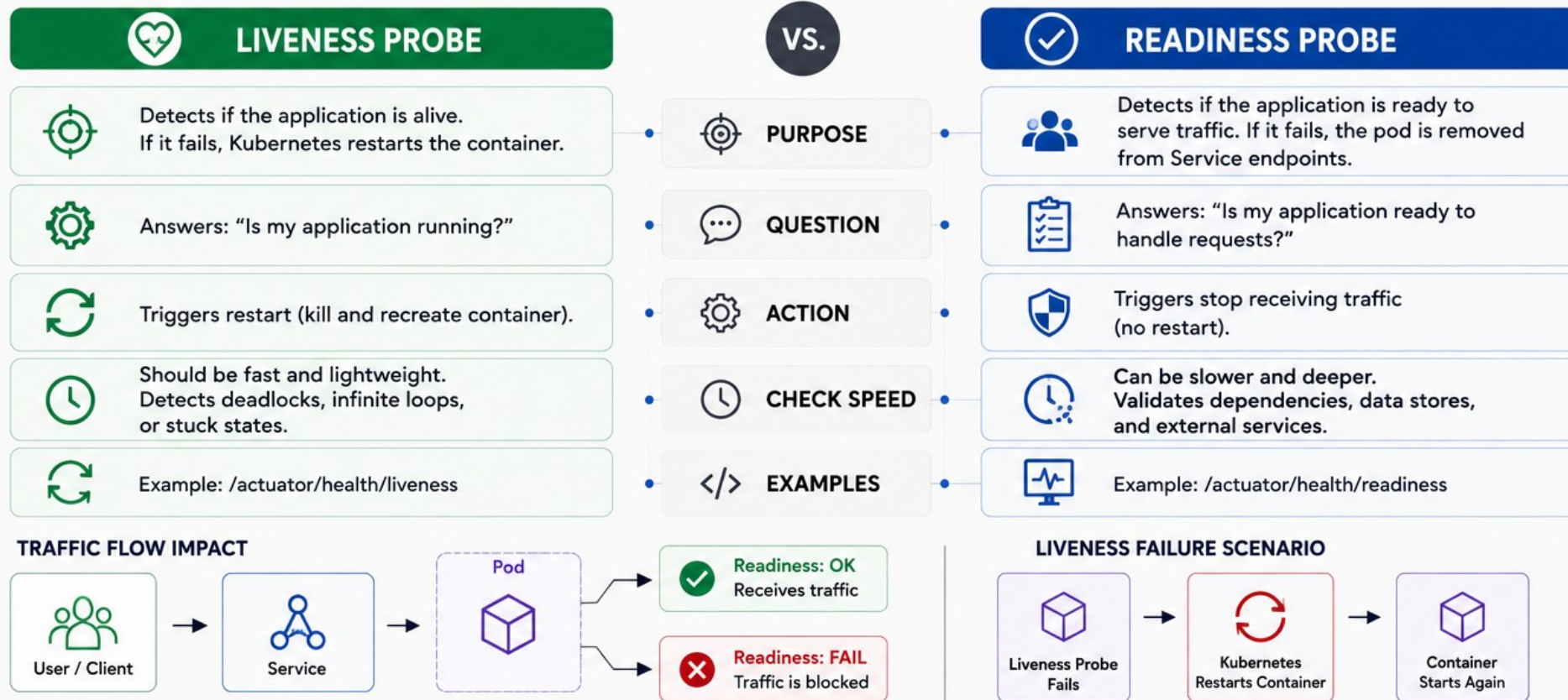
Metrics



Traces

Different signals with different purposes.

Together they keep your application healthy and your users happy.



KEY TAKEAWAY

Liveness keeps the container alive.
Readiness ensures only ready instances serve traffic.
Configure both probes thoughtfully for resilient, user-friendly applications.



BEST PRACTICES

- Keep liveness checks simple and fast.
- Make readiness checks meaningful and comprehensive.
- Use appropriate timeouts, initial delays, and failure thresholds.
- Test probe behavior under failure and recovery scenarios.



ALERTING PRINCIPLES

An alert tied to user impact or SLO burn, with enough context to act on immediately, is the bar every alerting rule should be held to.



SIX CORE ALERTING PRINCIPLES

- 1. Be specific:** say what broke and where, so nobody has to guess which service is involved
- 2. Be actionable:** every alert points at something a human can actually do right now
- 3. Be timely:** fire early enough to act, late enough to avoid noise from transient blips
- 4. Be relevant:** route to the team that owns the service, not to a channel everyone ignores
- 5. Be noise-free:** an alert that is routinely dismissed trains people to ignore the real one
- 6. Be reviewed:** prune and tune alerts regularly, because systems and thresholds drift



ALERT FLOW

- Metrics/Logs/Traces -> Detection -> Alert Triggered -> Notification -> Investigation & Action -> Review & Improve

KEY TAKEAWAY An alert tied to user impact or SLO burn, with enough context to act on immediately, is the bar every alerting rule should be held to.

REDUCING ALERT FATIGUE

Focusing on outcomes, deduplicating, and suppressing intelligently turn a wall of noisy alerts into a short list of actionable insights.



SIX STRATEGIES TO REDUCE FATIGUE

- 1. Focus on outcomes:** alert on user-visible symptoms rather than on every internal metric
- 2. Route and escalate:** send to the owning team first, with a defined path if nobody responds
- 3. Tune and test:** adjust thresholds against real incidents, and verify the alerts actually fire
- 4. Deduplicate:** group related alerts so one failure produces one page, not fifty
- 5. Suppress intelligently:** silence downstream alerts during a known upstream outage or deployment
- 6. Continuously improve:** feed every noisy page back into the alert definitions



COMMON CAUSES OF ALERT FATIGUE

- Too many alerts, alerting on symptoms instead of root causes, poorly tuned thresholds
- Duplicate/overlapping alerts and alerts not routed to the right team

KEY TAKEAWAY Focusing on outcomes, deduplicating, and suppressing intelligently turn a wall of noisy alerts into a short list of actionable insights.

OBSERVABILITY FOR MESSAGING SYSTEMS · OBSERVABILITY MATURITY MODEL



WHAT TO OBSERVE

- Health (broker availability, connectors), throughput (in/out rates)
- Backlog (queue depth, consumer lag), errors (DLQ rate, retries), latency (end-to-end)



COMMON ALERTS

- Consumer lag exceeds threshold; queue depth or oldest message age too high
- Error rate or DLQ messages detected; broker down or partition unavailable



THE FIVE MATURITY LEVELS

- **Level 1 Initial:** basic monitoring, manual troubleshooting
- **Level 2 Developing:** centralized logs, dashboards, defined alerting
- **Level 3 Defined:** distributed tracing, SLOs, error budgets, capacity visibility
- **Level 4 Advanced:** anomaly detection, predictive alerts, service dependency maps
- **Level 5 Optimized:** autonomous remediation, observability as a product

KEY TAKEAWAY Consumer lag and dead-letter-queue depth are the two messaging-specific signals that traditional API monitoring alone would never catch. • Level 1's manual troubleshooting evolves through correlation and prediction all the way to Level 5's autonomous remediation and self-service analytics.

PRODUCTION TROUBLESHOOTING WORKFLOW

A structured approach to quickly identify, diagnose, and resolve issues in your microservices systems.

Observability

The Three Pillars



Logs

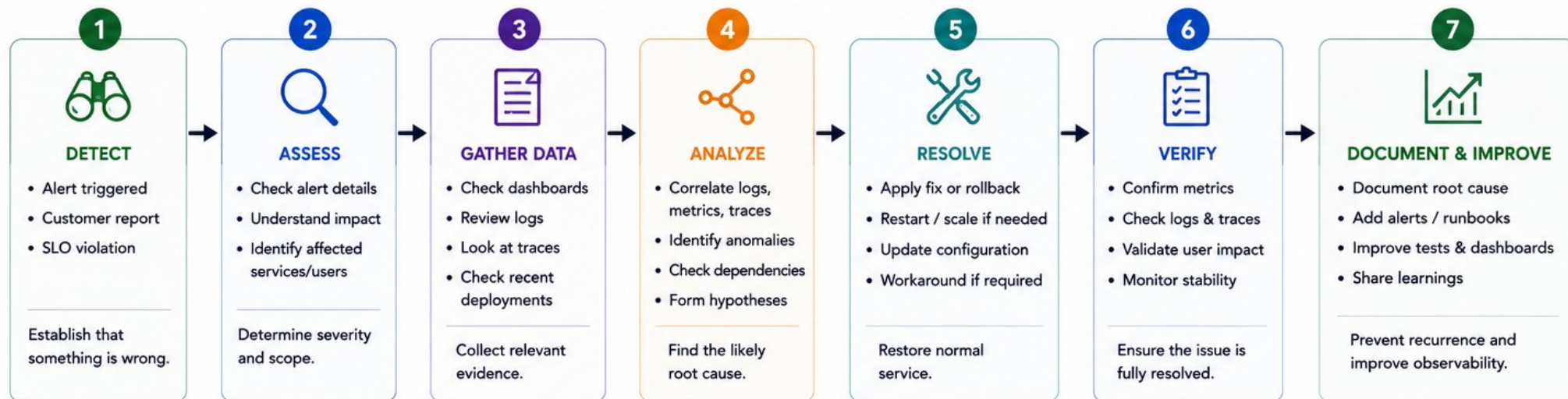


Metrics



Traces

TROUBLESHOOTING WORKFLOW



USE THE THREE PILLARS



LOGS

What happened?
Errors, exceptions, and events.



METRICS

What is the trend?
Rates, latency, saturation, errors.



TRACES

How does the request flow?
Follow the path and timings.



KEY QUESTIONS TO ASK



When did the issue start?



What changed recently?



Who or what is impacted?



Which service(s) are affected?



Is it isolated or widespread?



What is the user impact?



COMMON TOOLS



Grafana

Dashboards and metrics visualization



Prometheus

Metrics collection and alerting



ELK / Loki

Log aggregation and search



Jaeger / Tempo

Distributed tracing and analysis



Kubernetes / OpenShift

Pods, events, logs, and status



BEST PRACTICES

- Keep dashboards and alerts focused on outcomes.
- Correlate across logs, metrics, and traces.
- Use runbooks for common issues.
- Automate detection and recovery where possible.
- Communicate status clearly and frequently.
- Review incidents and improve continuously.



RUNBOOK ESSENTIALS



Symptoms



Diagnosis Steps



Resolution Steps



Verification Steps

EXERCISE 4.1 — ADD HEALTH PROBES AND CORRELATION LOGGING

Module 9 checkpoint A — Operability basics.



OBJECTIVE

- Operability basics
- 10 min · code warmup checkpoint
- **Worksheet:**
curriculum/day-04/exercises/exercise-4.1-probes-correlation.md



DO THIS

1. Liveness vs readiness vs startup — which Actuator paths, and why startup exists
2. X-Correlation-Id on the request must appear as correlationId= in logs
3. Health stays public; /actuator/metrics still needs a JWT
4. Tracing (OpenTelemetry) is a demonstration — you do not stand up Jaeger



EXPECTED RESULT

- You can tell liveness from readiness and find a correlation id in a log line.
- Lab 4 verifies liveness/readiness and correlationId= in logs, then the same probes on OpenShift.

KEY TAKEAWAY You can tell liveness from readiness and find a correlation id in a log line.